

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

Enhanced (Extended)ER (EER) model

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex queries, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF),

Week 7,8 : BCNF, introduction to the higher normal forms.

Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.

Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.

Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.

Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.

Week 14: Current trends in database system, introduction to data warehousing, data mining.

Practical: MySQL commands

Project: Team of 2 or 3 members

Enhanced (Extended)ER (EER) model

ER Model: for traditional database applications, which include many data-processing applications in business and industry.

EER Model: newer applications of database technology, such as databases for engineering design and manufacturing (CAD/CAM), telecommunications, complex software systems, and geographic information systems (GISs), AI applications, object modeling in software engineering domains.

EER Model Concepts:

- Includes all modeling concepts of basic ER
- Additional concepts:
 - subclasses/superclasses

Enhanced (Extended)ER model (EER) model

- specialization/generalization
 - categories (UNION types)
 - attribute and relationship inheritance
 - some object-oriented concepts, such as inheritance
 - knowledge Representation and Ontology Concepts
-
- Constraints on Specialization/Generalization

The additional EER concepts are used to model applications more completely and more accurately.

Subclasses and Superclasses or supertype/subtype or class/subclass relationship

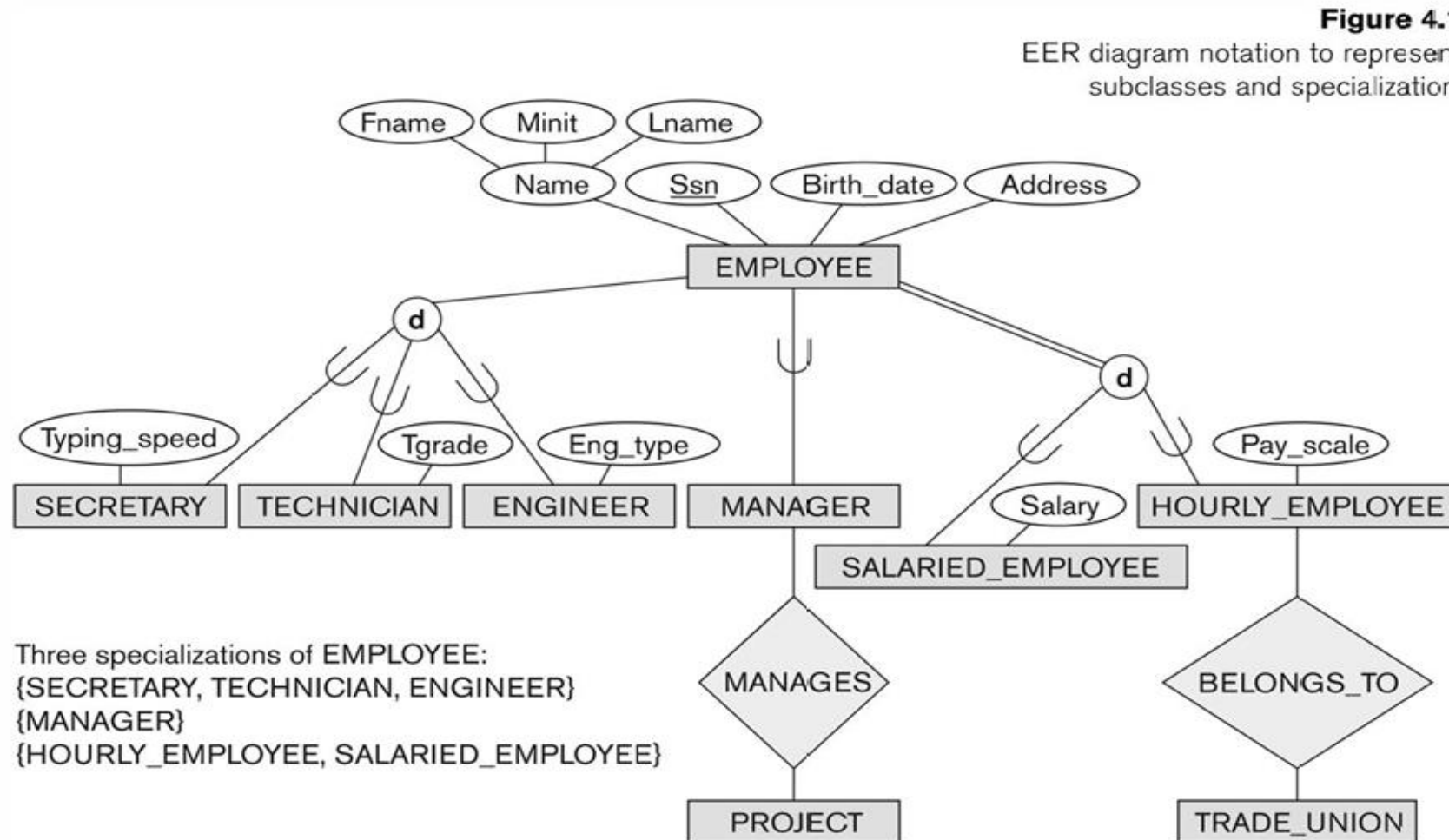
- An entity type may have additional meaningful subgroupings of its entities.
 - name of an entity type is used to represent both a type of entity and the entity set or collection of entities of that type that exist in the database.
 - Example: **EMPLOYEE** may be further grouped into:
 - SECRETARY, ENGINEER, TECHNICIAN.....etc., Based on the EMPLOYEE's Job.
 - **MANAGER**
 - EMPLOYEEs who are managers (the role they play)
 - **SALARIED_EMPLOYEE, HOURLY_EMPLOYEE**
 - Based on the EMPLOYEE's method of pay

Subclasses and Superclasses or supertype/subtype or class/subclass relationship

- EER diagrams extend ER diagrams to represent these additional subgroupings, called subclasses or subtypes.
- We call each of these subgroupings a **subclass or subtype** of the **EMPLOYEE** entity type, and the **EMPLOYEE** entity type is called the **superclass or supertype** for each of these subclasses.

Subclasses and Superclasses or supertype/subtype or class/subclass relationship

Subclasses and Superclasses



Subclasses and Superclasses (2)

- Each of these subgroupings is a subset of EMPLOYEE entities
- Each is called a subclass of EMPLOYEE
- EMPLOYEE is the superclass for each of these subclasses
- These are called superclass/subclass relationships:
 - EMPLOYEE/SECRETARY
 - EMPLOYEE/TECHNICIAN
 - EMPLOYEE/MANAGER
 - ...

Subclasses and Superclasses (3)

- These are also called IS-A relationships
 - SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE,
- Note: An entity that is member of a subclass represents the same real-world entity as some member of the superclass:
 - The subclass member is the same entity in a *distinct specific role*
 - An entity cannot exist in the database merely by being a member of a subclass; it must also be a member of the superclass
 - A member of the superclass can be optionally included as a member of any number of its subclasses

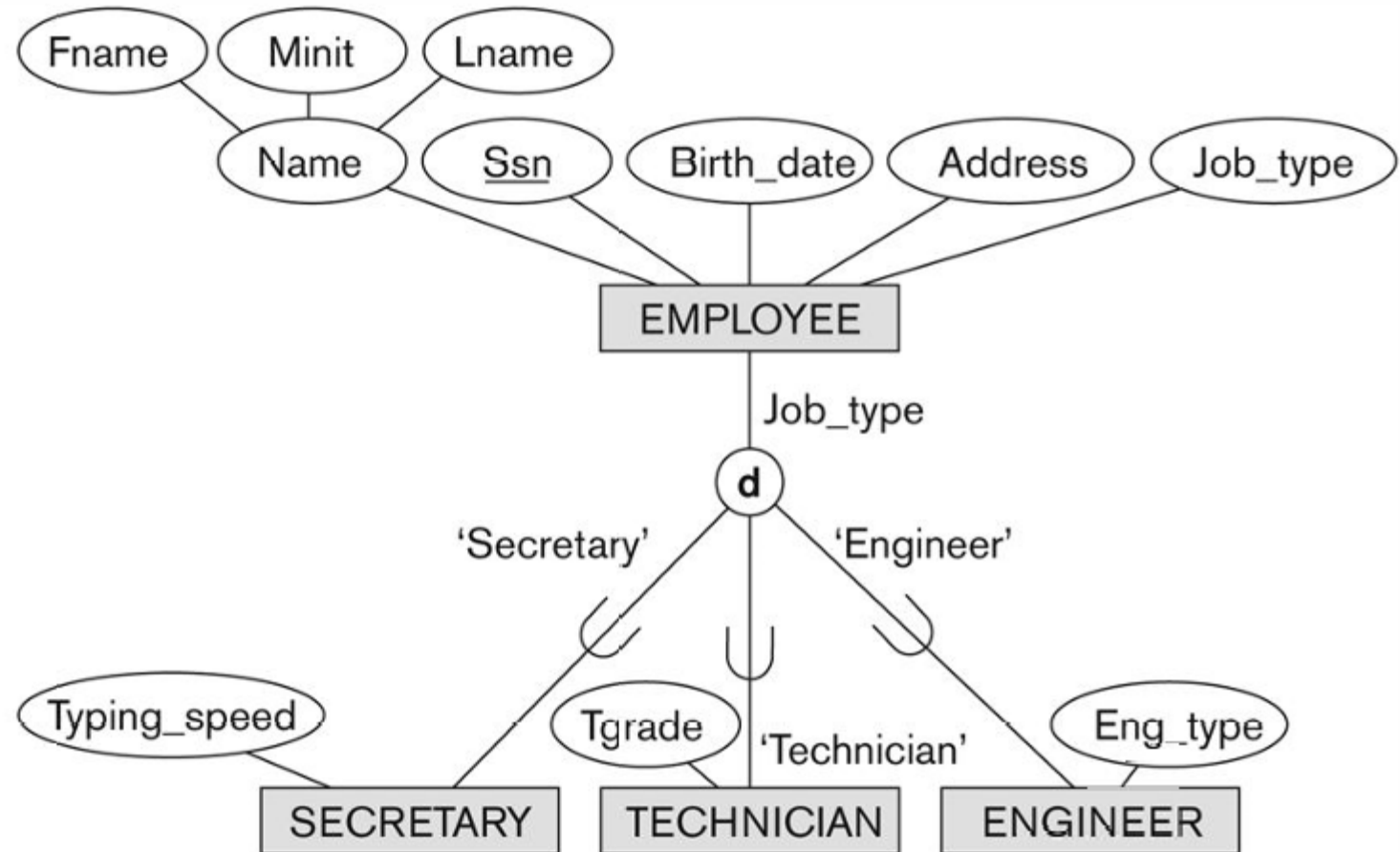
Subclasses and Superclasses (4)

- Examples:
 - A salaried employee who is also an engineer belongs to the two subclasses:
 - ENGINEER, and
 - SALARIED_EMPLOYEE
 - A salaried employee who is also an engineering manager belongs to the three subclasses:
 - MANAGER,
 - ENGINEER, and
 - SALARIED_EMPLOYEE
- It is not necessary that every entity in a superclass be a member of some subclass

Representing Specialization in EER Diagrams

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



Attribute Inheritance in Superclass / Subclass Relationships

- An entity that is member of a subclass *inherits*
 - All attributes of the entity as a member of the superclass
 - All relationships of the entity as a member of the superclass
- Example:
 - In the previous slide, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, ..., from EMPLOYEE
 - Every SECRETARY entity will have values for the inherited attributes

Specialization and Generalization:

Specialization: It's a **top-down approach** where a higher-level entity is broken down into more specialized entities.

e.g.: **EMPLOYEE** ----→ **ENGINEER**
MANAGER
TECHNICIAN
SALERIED EMPLOYEE

Generalization: It's a **bottom-up approach** where multiple specific entities are generalized into a single, more abstract entity.

e.g.: **CAR**
BUS -----→ **VEHICLE**
TRUCK

Specialization (1)

- Specialization is the process of defining a set of subclasses of a superclass
- The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass
 - Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon *job type*.
 - Example: MANAGER *is a specialization of EMPLOYEE based on the role the employee plays*
 - May have several specializations of the same superclass

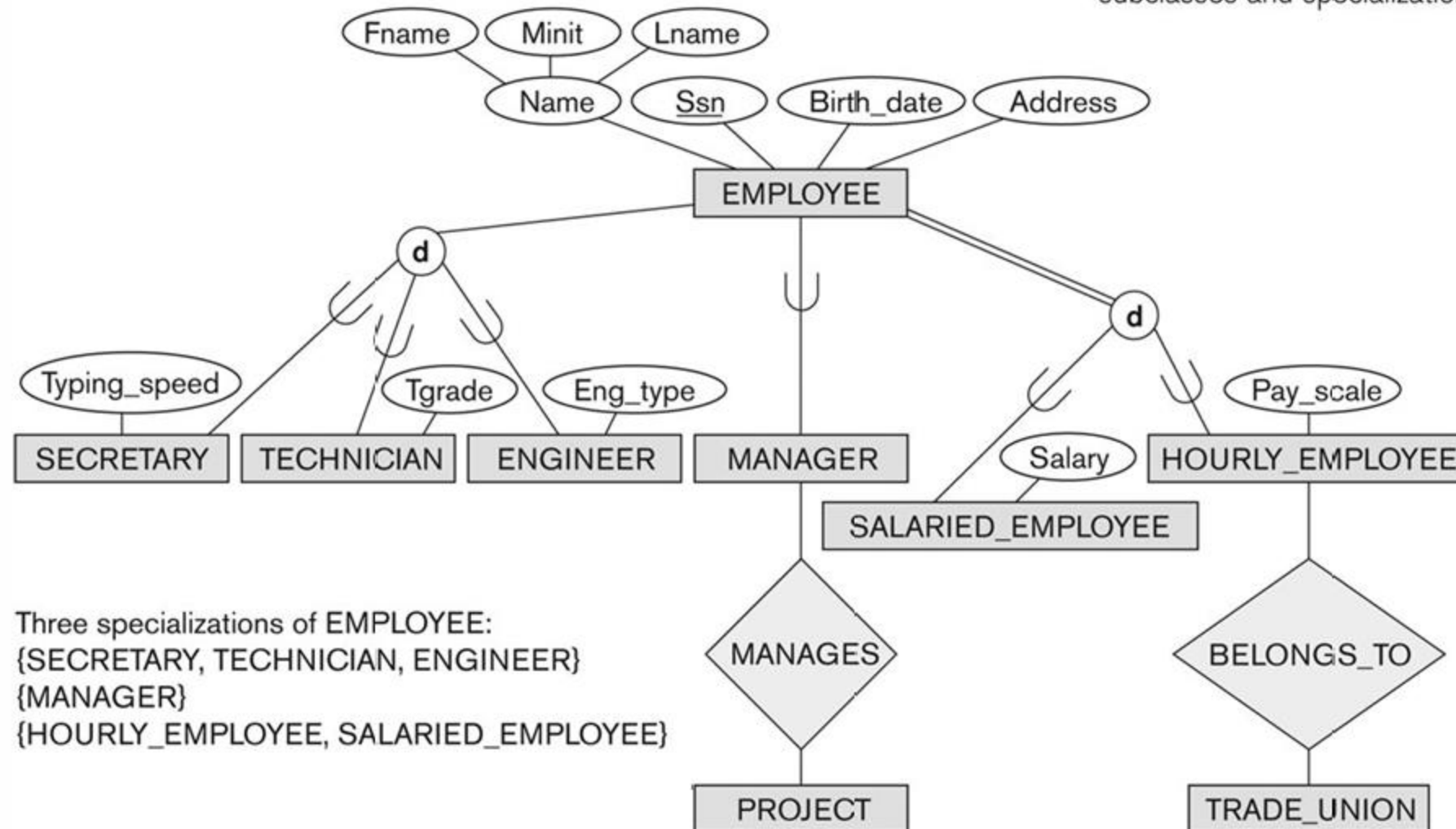
Specialization (2)

- Example: Another specialization of EMPLOYEE based on *method of pay* is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.
 - Superclass/subclass relationships and specialization can be diagrammatically represented in EER diagrams
 - Attributes of a subclass are called *specific* or *local* attributes.
 - For example, the attribute TypingSpeed of SECRETARY
 - The subclass can also participate in specific relationship types.
 - For example, a relationship BELONGS_TO of HOURLY_EMPLOYEE

Specialization (3)

Figure 4.1

EER diagram notation to represent subclasses and specialization.



Generalization

- Generalization is the reverse of the specialization process
- Several classes with common features are generalized into a superclass;
 - original classes become its subclasses
- Example: CAR, TRUCK generalized into VEHICLE;
 - both CAR, TRUCK become subclasses of the superclass VEHICLE.
 - We can view {CAR, TRUCK} as a specialization of VEHICLE
 - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK

Generalization (2)

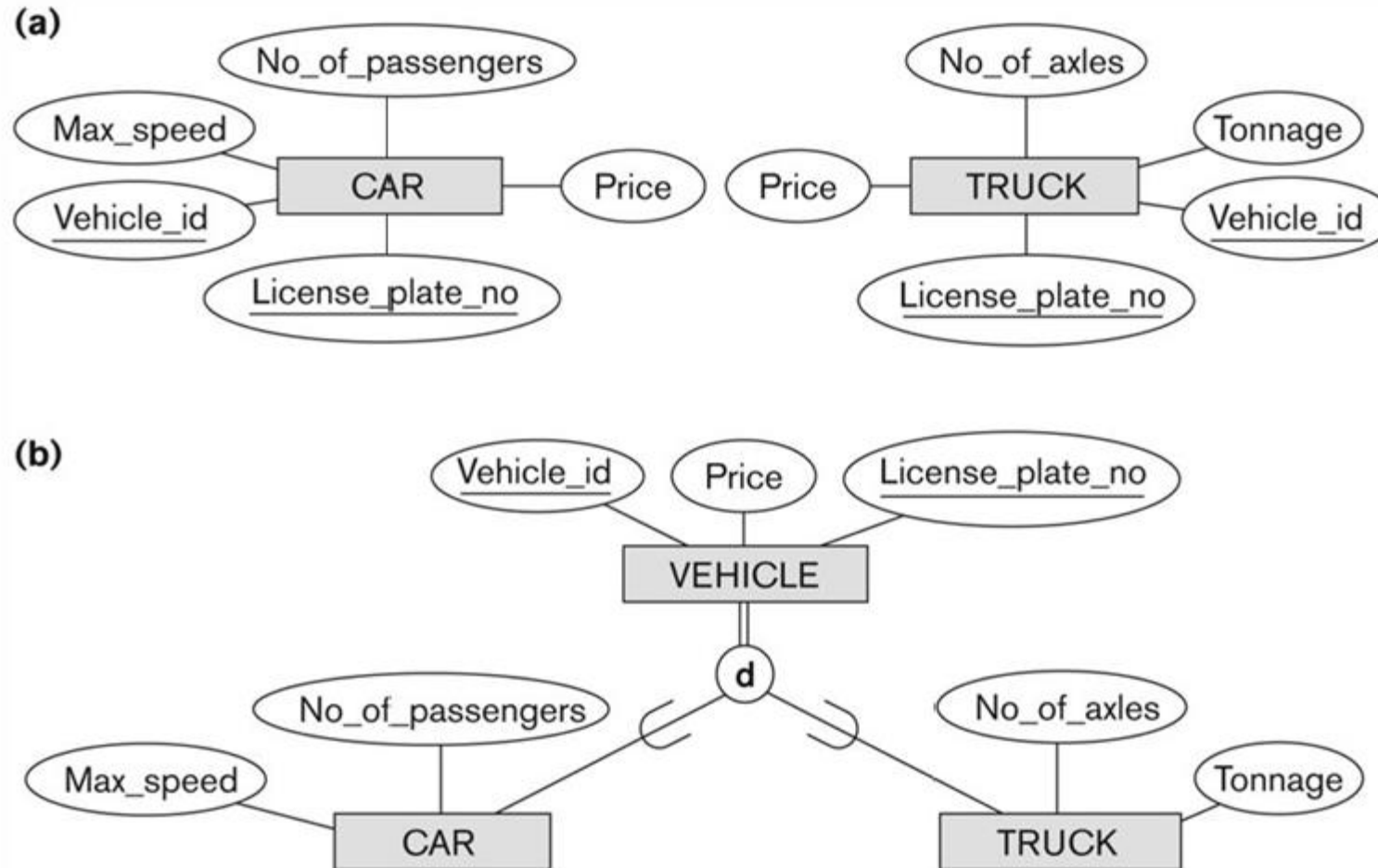


Figure 4.3

Generalization. (a) Two entity types, CAR and TRUCK. (b) Generalizing CAR and TRUCK into the superclass VEHICLE.

Generalization and Specialization (1)

- Diagrammatic notations are sometimes used to distinguish between generalization and specialization
 - Arrow pointing to the generalized superclass represents a generalization
 - Arrows pointing to the specialized subclasses represent a specialization
 - *We do not use* this notation because it is often subjective as to which process is more appropriate for a particular situation
 - We advocate not drawing any arrows

Instead, EER uses other clear and standardized notations (like circles, triangles, or lines) to represent hierarchical relationships like specialization and generalization.

Generalization and Specialization (2)

- Data Modeling with Specialization and Generalization
 - A superclass or subclass represents a collection (or set or grouping) of entities
 - It also represents a particular *type of entity*
 - Shown in rectangles in EER diagrams (as are entity types)
 - We can call all entity types (and their corresponding collections) **classes**, whether they are entity types, superclasses, or subclasses

Types of Specialization

- Predicate-defined (or condition-defined) : based on some predicate. E.g., based on value of an attribute, say, Job-type, or Age.
- Attribute-defined: shows the name of the attribute next to the line drawn from the superclass toward the subclasses (see Fig. 4.1)
- User-defined: membership is defined by the user on an entity by entity basis

Predicate is a logical condition or constraint.

Constraints on Specialization and Generalization (1)

- If we can determine exactly those entities that will become members of each subclass by a condition, the subclasses are called predicate-defined (or condition-defined) subclasses
 - Condition is a constraint that determines subclass members
 - Display a predicate-defined subclass by writing the predicate condition next to the line attaching the subclass to its superclass

→ We define the specialization based on specific conditions or predicates that apply to the `job_title` attribute or to any other attributes.

Constraints on Specialization and Generalization (2)

- If all subclasses in a specialization have membership condition on same attribute of the superclass, specialization is called an attribute-defined specialization
 - Attribute is called the defining attribute of the specialization
 - Example: JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE
- If no condition determines membership, the subclass is called user-defined
 - Membership in a subclass is determined by the database users by applying an operation to add an entity to the subclass
 - Membership in the subclass is specified individually for each entity in the superclass by the user

In attribute-defined specialization, an entity is divided into subtypes based on the **value** of a particular attribute.

e.g.: The Vehicle entity is divided into different subtypes like Car, Truck, and Motorcycle, based on an **attribute** such as vehicle_type.

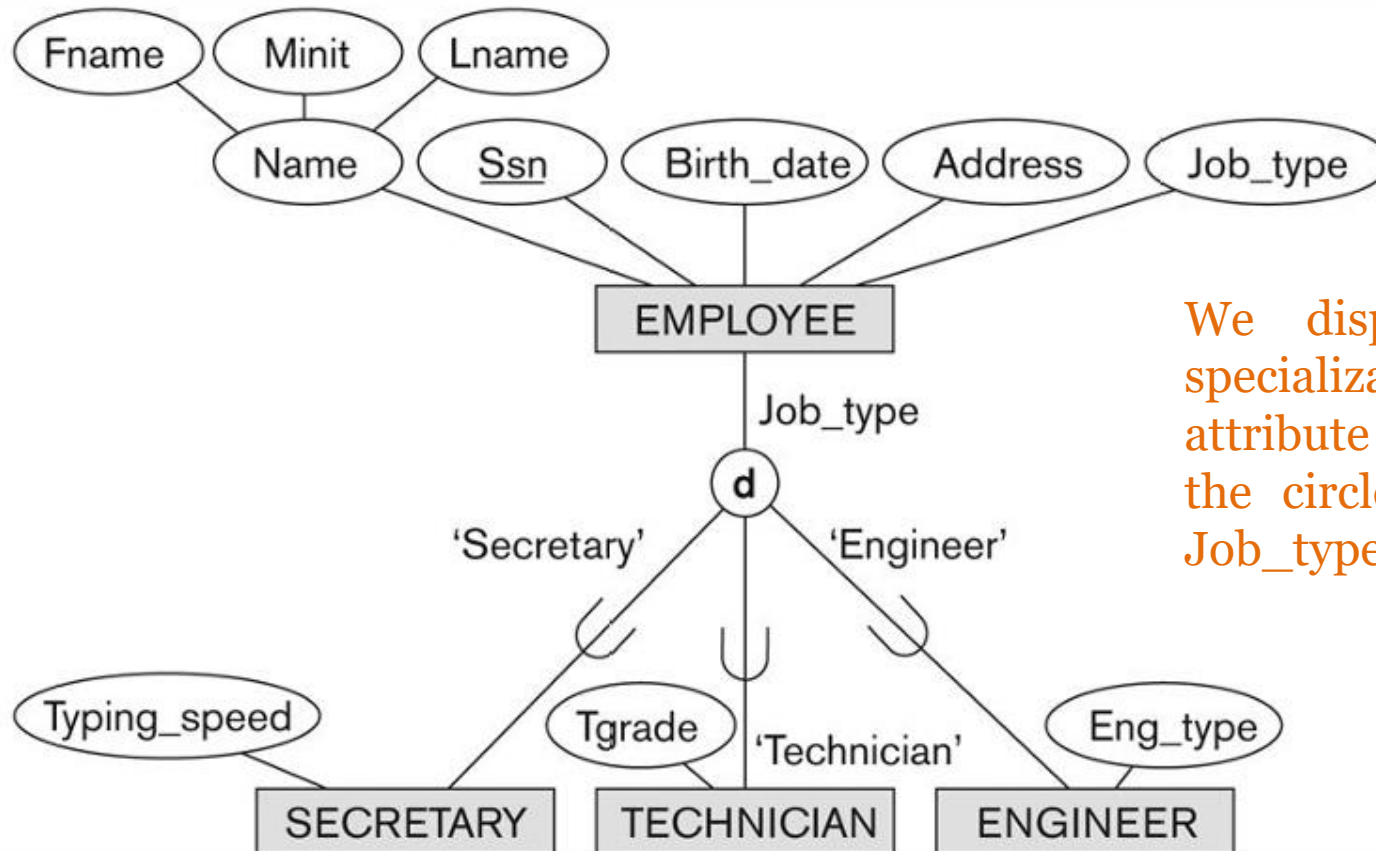
❑ User defined Type of Specialization

- In user-defined specialization, the specialization of an entity into subtypes is based on **user-defined conditions** rather than *pre-defined attributes or logical predicates*.
- These conditions could be based on the user's specific needs, business rules, or additional criteria that ***aren't necessarily represented as attributes*** in the entity itself.
- e.g. : PERSON entity is specialized into STUDENT and FACULTY based on the user's definition: enrollment in courses (for students) and holding a faculty position (for faculty members).

Displaying an attribute-defined specialization in EER diagrams

Figure 4.4

EER diagram notation for an attribute-defined specialization on Job_type.



We display an attribute-defined specialization by placing the defining attribute name next to the arc from the circle to the superclass. Here, **Job_type** is such an attribute.

Constraints on Specialization and Generalization (3)

- Two basic constraints can apply to a specialization/generalization:
 - Disjointness Constraint:
 - Completeness Constraint:

Constraints on Specialization and Generalization (4)

■ Disjointness Constraint:

- Specifies that the subclasses of the specialization must be *disjoint*:

- an entity can be a member of at most one of the subclasses of the specialization

- Specified by d in EER diagram

- If not disjoint, specialization is *overlapping*:

- that is the same entity may be a member of more than one subclass of the specialization

- Specified by o in EER diagram

Disjoint indicates that an entity can belong to **at most one subtype** within the hierarchy.

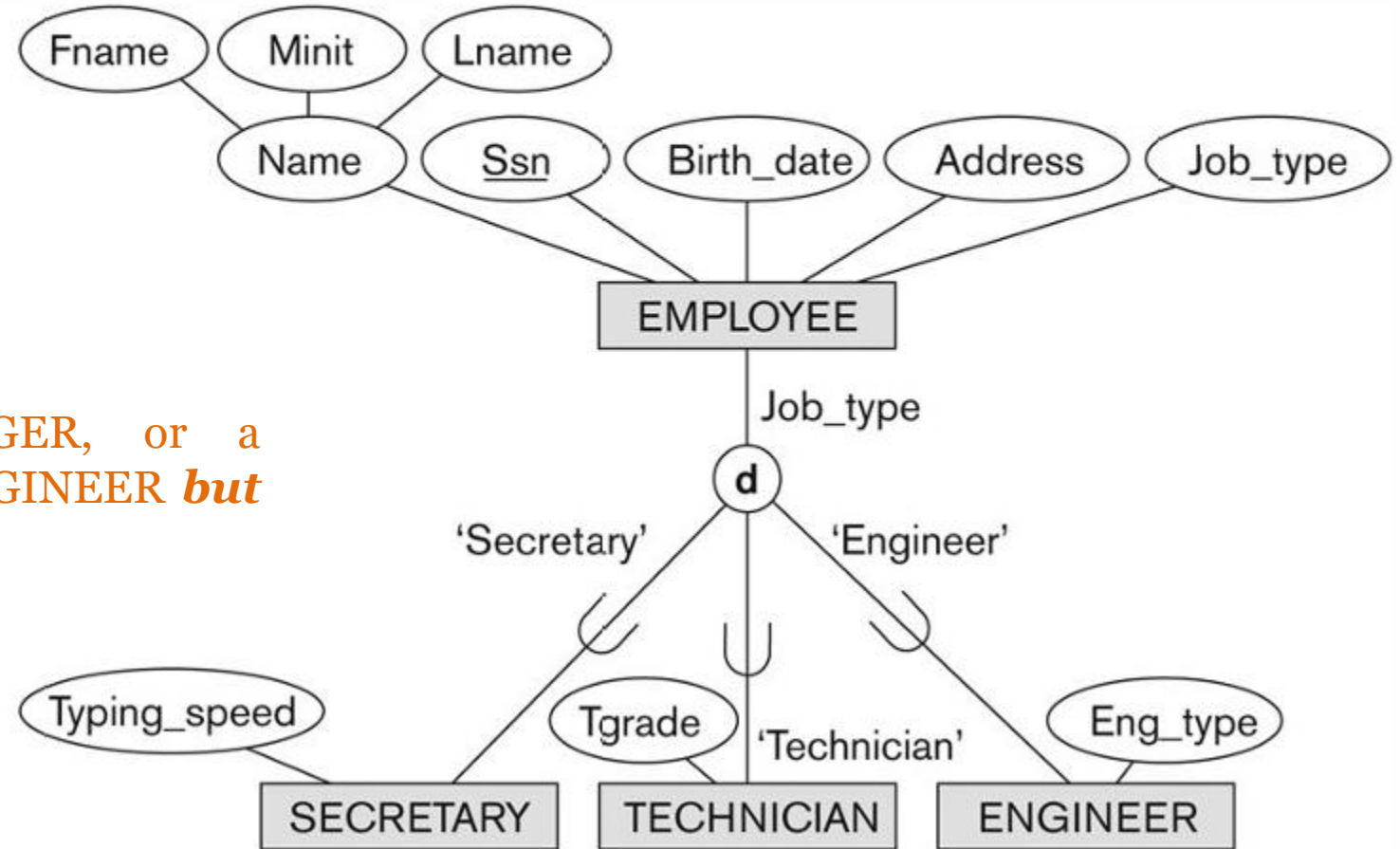
Overlap indicates that entity instance can belong to **multiple subtypes simultaneously**. (e.g., an employee can be both a manager and a technician at the same time).

Constraints on Specialization and Generalization (6)

- Hence, we have four types of specialization/generalization:
 - Disjoint, total
 - Disjoint, partial
 - Overlapping, total
 - Overlapping, partial
- Note: Generalization usually is total because the superclass is derived from the subclasses.

Example of disjoint partial Specialization

Figure 4.4
EER diagram notation
for an attribute-
defined specialization
on Job_type.



EMPLOYEE can be either a MANAGER, or a SECRETARY, or a TECHNICIAN, or an ENGINEER **but not more than one** at the same time.

Example of overlapping total Specialization

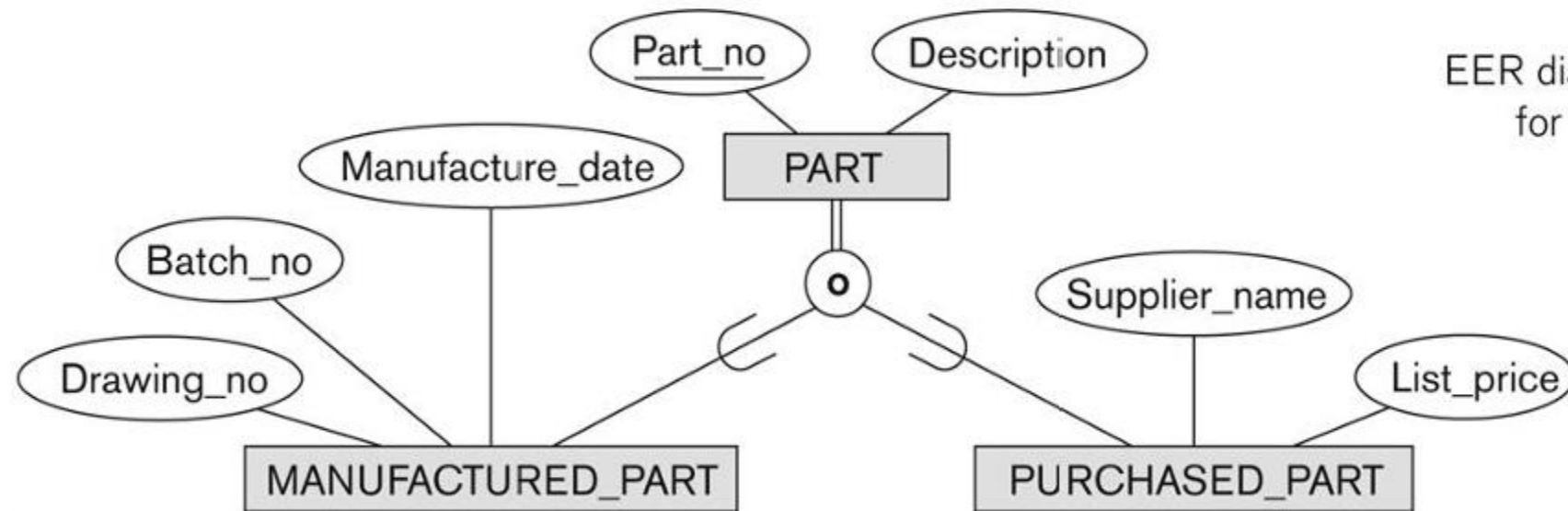


Figure 4.5
EER diagram notation
for an overlapping
(nondisjoint)
specialization.

Alternative diagrammatic notations

- ER/EER diagrams are a specific notation for displaying the concepts of the model diagrammatically
- DB design tools use many alternative notations for the same or similar concepts
- One popular alternative notation uses *UML class diagrams*
- see next slides for UML class diagrams and other alternative notations

UML Example for Displaying Specialization / Generalization

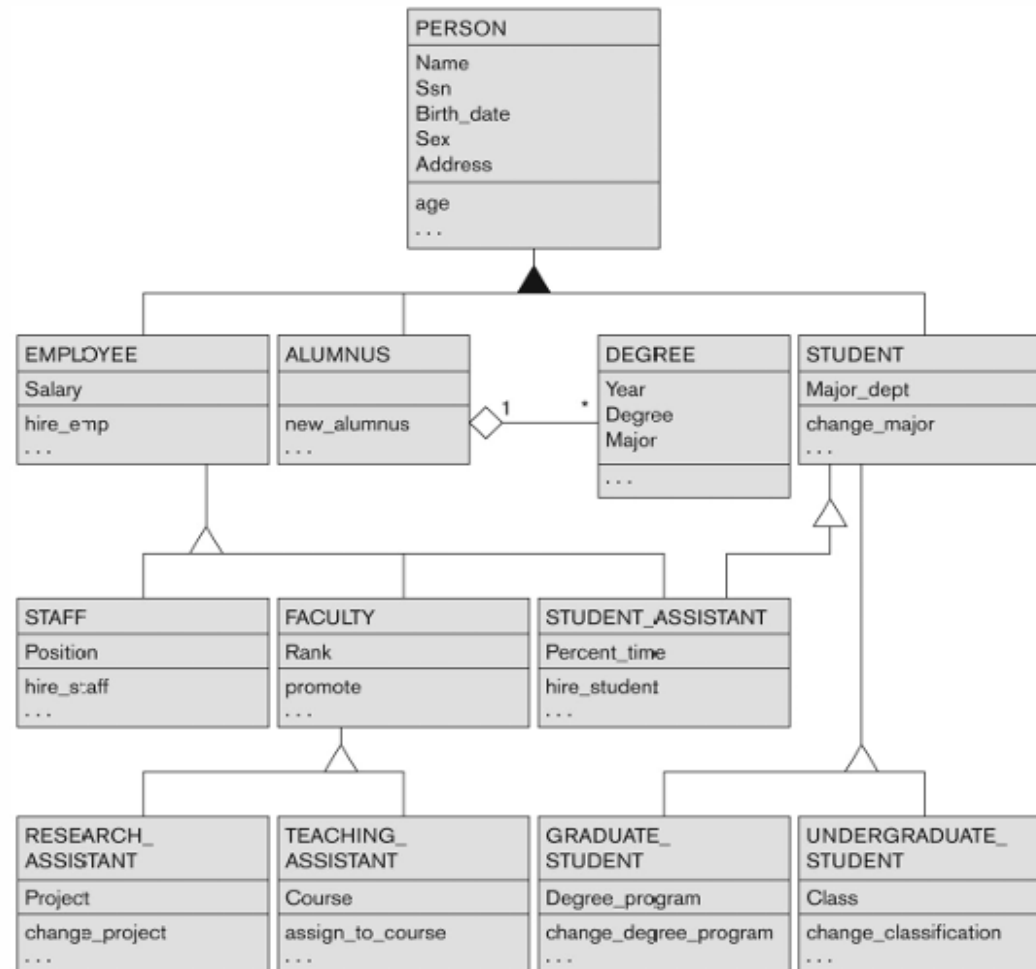


Figure 4.10

A UML class diagram corresponding to the EER diagram in Figure 4.7, illustrating UML notation for specialization/generalization.

Alternative Diagrammatic Notations

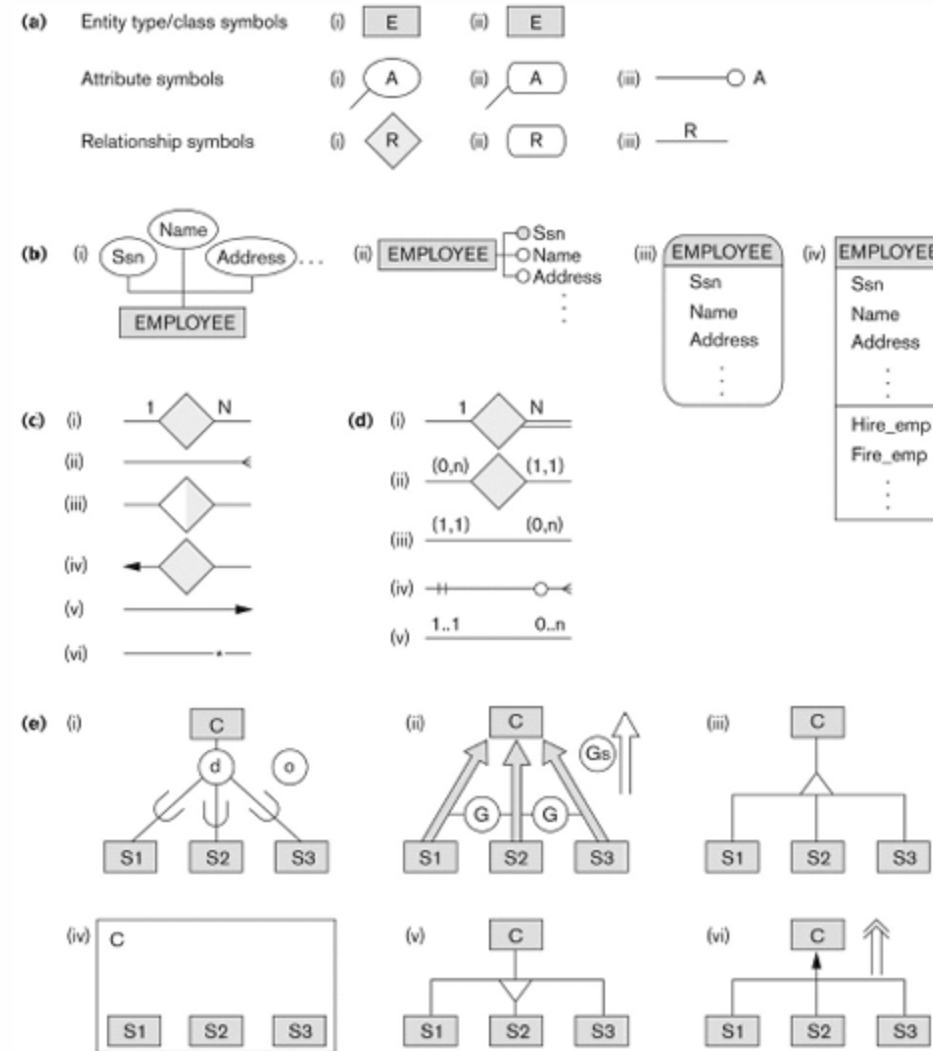


Figure A.1

Alternative notations. (a) Symbols for entity type/class, attribute, and relationship. (b) Displaying attributes. (c) Displaying cardinality ratios. (d) Various (min, max) notations. (e) Notations for

THANK YOU